

Mobius

莫比乌斯反演

引入

莫比乌斯反演是数论中的重要内容。对于一些函数，如果很难直接求出它的值，而容易求出其倍数和或约数和，那么可以通过莫比乌斯反演简化运算，求得 的值。

开始学习莫比乌斯反演前，需要先学习一些前置知识：[数论分块](#)、[狄利克雷卷积](#)。

莫比乌斯函数

定义

为莫比乌斯函数，定义为

含有平方因子 为的本质不同质因子个数

详细解释一下：

令，其中 为质因子，。上述定义表示：

- 1. 时，；
- 2. 对于 时：
 - 1. 当存在，使得 时，，也就是说只要某个质因子出现的次数超过一次，就等于；
 - 2. 当任意，都有 时，，也就是说每个质因子都仅仅只出现过一次时，即，中个元素唯一时，等于 的 次幂，此处 指的便是仅仅只出现过一次的质因子的总个数。

性质

莫比乌斯函数不仅是积性函数，还有如下性质：

即，

证明

设

那么

根据二项式定理，易知该式子的值在 即 时值为 否则为，这也同时证明了 以及

▼ 注

这个性质意味着，莫比乌斯函数在狄利克雷生成函数中，等价于黎曼函数 的倒数。所以在狄利克雷卷积中，莫比乌斯函数是常数函数 的逆元。

补充结论

反演结论：

直接推导：如果看懂了上一个结论，这个结论稍加思考便可以推出：如果 $\mu(n) \neq 0$ 的话，那么代表着我们按上个结论中枚举的那个 d 是 n 的约数，也就是式子的值是 $\mu(n/d)$ ，反之，有一个与 n 相同的值：

利用 函数：根据上一结论， $\mu(n) = \sum_{d|n} \mu(d)$ ，将 $\mu(n)$ 展开即可。

线性筛

由于 μ 函数为积性函数，因此可以线性筛莫比乌斯函数（线性筛基本可以求所有的积性函数，尽管方法不尽相同）。

▼ 线性筛实现



C++Python

```
1 void getMu() {
2     mu[1] = 1;
3     for (int i = 2; i <= n; ++i) {
4         if (!flg[i]) p[++tot] = i, mu[i] = -1;
5         for (int j = 1; j <= tot && i * p[j] <= n; ++j) {
6             flg[i * p[j]] = 1;
7             if (i % p[j] == 0) {
8                 mu[i * p[j]] = 0;
9                 break;
10            }
11            mu[i * p[j]] = -mu[i];
12        }
13    }
14 }
```

```
1 def getMu():
2     mu[1] = 1
3     for i in range(2, n + 1):
4         if flg[i] != 0:
5             p[tot] = i; tot = tot + 1; mu[i] = -1
6             j = 1
7             while j <= tot and i * p[j] <= n:
8                 flg[i * p[j]] = 1
9                 if i % p[j] == 0:
10                     mu[i * p[j]] = 0
11                     break
12                 mu[i * p[j]] = mu[i * p[j]] - mu[i]
13                 j = j + 1
```

拓展

证明

将 n 分解质因数：

首先，因为 μ 是积性函数，故只要证明当 n 为质数时成立即可。

因为 p 是质数，于是

易知如下过程：

该式子两侧同时卷 可得

莫比乌斯变换

设 为两个数论函数。

形式一：如果有 ，那么有 。

这种形式下，数论函数 称为数论函数 的莫比乌斯变换，数论函数 称为数论函数 的莫比乌斯逆变换（反演）。

容易看出，数论函数 的莫比乌斯变换，就是将数论函数 与常数函数 进行狄利克雷卷积。

▼ 注

根据狄利克雷卷积与狄利克雷生成函数的对应关系，数论函数 的莫比乌斯变换对应的狄利克雷生成函数，就是数论函数 的狄利克雷生成函数与黎曼函数 的乘积。

形式二：如果有 ，那么有 。

证明

方法一：对原式做数论变换。

用 来替换，再变换求和顺序。最后一步变换的依据：，因此在 时第二个和式的值才为 。此时，故原式等价于

方法二：运用卷积。

原问题为：已知，证明

易知如下转化：（其中）。

对于第二种形式：

类似上面的方法一，我们考虑逆推这个式子。

我们把 表示为 的形式，然后把 的原定义代入式子。

发现枚举 再枚举 的倍数可以转换为直接枚举 的倍数再求出，发现后面那一块其实就是，整个式子只有在 的时候才能取到值。

问题形式

[「HAOI 2011」Problem b](#)

求值（多组数据）

根据容斥原理，原式可以分成 块来处理，每一块的式子都为

考虑化简该式子

因为 i 对答案才有贡献，于是我们可以将其替换为 $\begin{cases} i & \text{当且仅当 } i \text{ 值为 } 1 \\ 0 & \text{否则为 } 0 \end{cases}$ ，故原式化为

将 f 函数展开得到

变换求和顺序，先枚举 j 可得

易知 n 中 j 的倍数有 $\frac{n}{j}$ 个，故原式化为

很显然，式子可以数论分块求解。

时间复杂度

► 代码实现

[「SPOJ 5971」LCMSUM](#)

求值（多组数据）

易得原式即

将原式复制一份并且颠倒顺序，然后将 n 一项单独提出，可得

根据 $\sum_{d|n} \frac{n}{d} = \sum_{d|n} d$ ，可将原式化为

两个求和式中分母相同的项可以合并。

即

可以将相同的 $\frac{n}{d}$ 合并在一起计算，故只需要统计 $\frac{n}{d}$ 的个数。当 $n=1$ 时， $\frac{n}{d}=1$ ，所以 $\frac{n}{d}$ 的个数有 1 个。

故答案为

变换求和顺序，设 $S(n) = \sum_{d|n} d$ ，合并公因式，式子化为

设 μ 为积性函数，于是可以筛出 μ 。每次询问 $\sum_{d|n} \mu(d)$ 计算即可。

下面给出这个函数筛法的推导过程：

首先考虑 $\phi(n)$ 的值，显然它的约数只有 $1, 2, 4$ ，因此

又有 $\phi(n) \mid n$ ，则原式可化为

于是有

那么，对于线性筛中的 i ，令 $j = \frac{n}{i}$ ，可得

即

同理有

因此

时间复杂度：

► 代码实现

[「BZOJ 2154」Crash 的数字表格](#)

求值（对 10^9+7 取模）

易知原式等价于

枚举最大公因数 d ，显然两个数除以 d 得到的数互质

非常经典的 ϕ 式子的化法

后半段式子中，出现了互质数对之积的和，为了让式子更简洁就把它拿出来单独计算。于是我们记

接下来对 S 进行化简。首先枚举约数 d ，并将 S 表示为

设 $i = dx$ ， $j = dy$ ，显然式子可以变为

观察上式，前半段可以预处理前缀和；后半段又是一个范围内数对之和，记

可以 求解

至此

我们可以 数论分块求解 函数。

在求出 后，回到定义 的地方，可得原式为

可见这又是一个可以数论分块求解的式子！

本题除了推式子比较复杂、代码细节较多之外，是一道很好的莫比乌斯反演练习题！（上述过程中，默认）

时间复杂度：（瓶颈为线性筛）

► 代码实现

[「SDOI2015」约数个数和](#)

多组数据，求

其中， $d(n)$ 表示 n 的约数个数

要推这道题首先要了解 函数的一个特殊性质

再化一下这个式子

将上述式子代回原式

那么 预处理 的前缀和，分块处理询问，总复杂度。

► 代码实现

莫比乌斯反演扩展

结尾补充一个莫比乌斯反演的非卷积形式。

对于数论函数 f 和完全积性函数 g 且：

我们证明一下

【先枚举乘积】 【对答案的贡献为，于是省略】 【是完全积性函数】 【】 【当且仅当时】

参考文献

[algocode 算法博客](#)

本页面最近更新：2023/10/4 21:50:08, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[ranwen](#), [StudyingFather](#), [383494](#), [H-J-Granger](#), [hyp1231](#), [iamtwz](#), [Marcythm](#), [Peanut-Tang](#), [Chrogeek](#), [countercurrent-time](#), [CyaceQuious](#), [Early0v0](#), [Enter-tainer](#), [ezoixx130](#), [FFjet](#), [frank-xjh](#), [GekkaSaori](#), [Gesruea](#), [Great-designer](#), [guodong2005](#), [hehelego](#), [Henry-ZHR](#), [hjsjhn](#), [hydingsy](#), [i-yyi](#), [lr1d](#), [kenlig](#), [ksyx](#), [Lcyanstars](#), [Luckyblock233](#), [luojiny1](#), [MegaOwler](#), [Menci](#), [mgt](#), [mxr612](#), [NachtgeistW](#), [nalemy](#), [orzAtalod](#), [ouuan](#), [qwqAutomaton](#), [SamZhangQingChuan](#), [ShaoChenHeng](#), [shawlleyw](#), [Siyuan](#), [Sshwy](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [UserUnauthorized](#), [Vxlimo](#), [WineChord](#), [Xeonacid](#), [yjl9903](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用